

Data Migration and Resilience



Introduction to Data Migration



Engage and Think



You are overseeing a critical data migration from an on-premises system to a new environment. Shortly after cutover, you discover that a core production table is missing several records in the target database, even though they exist in the source. Business operations are already live, and downtime is costly.

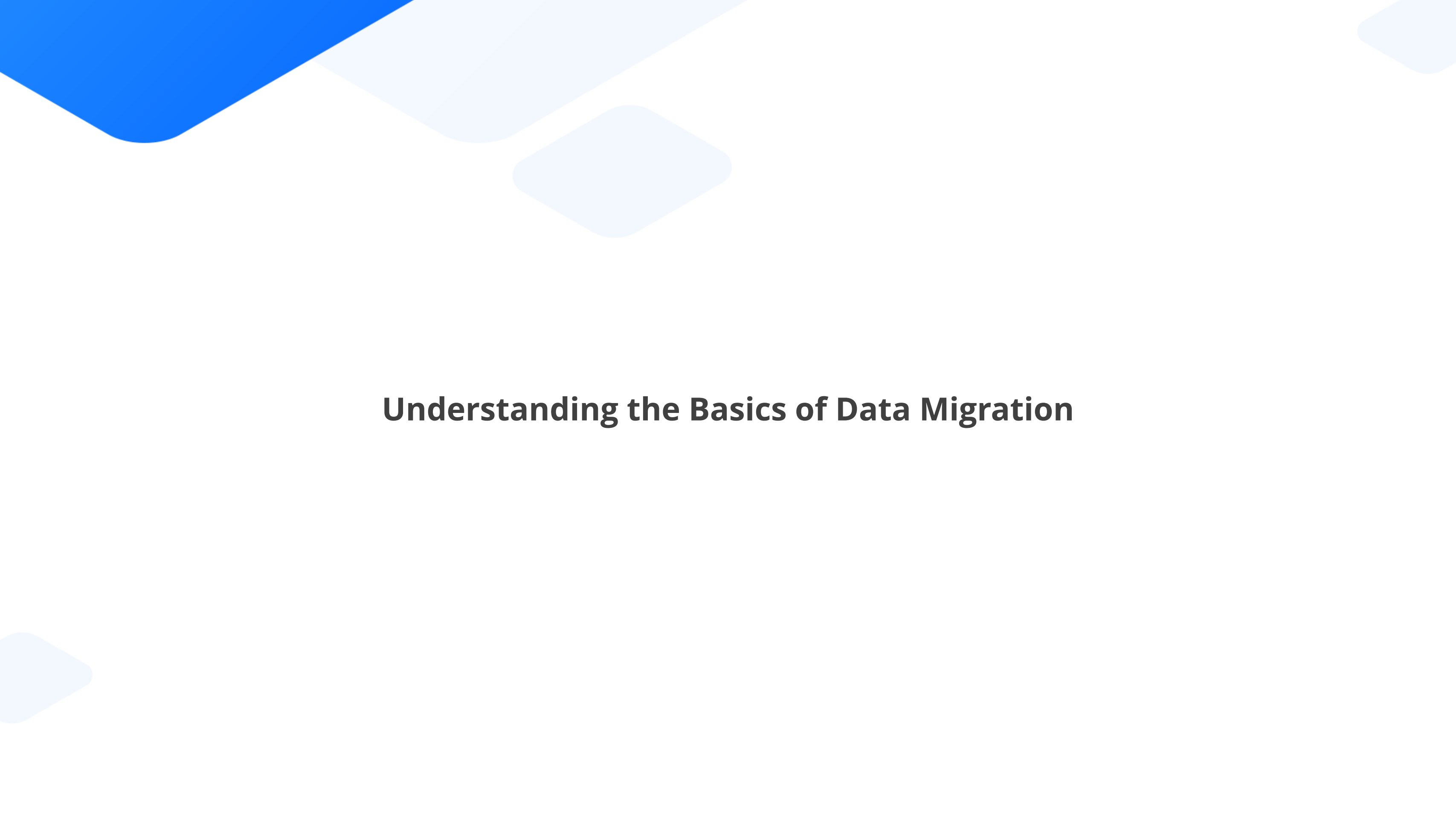
What is the first step you should take?

Learning Objectives

By the end of this lesson, you will be able to:

- 🔗 Define the types of data migration, such as storage, database, and application or replication, and differentiate between traditional and cloud-based approaches
- 🔗 Explain key considerations in cloud data migration, including compliance, security, network latency, and cost optimization
- 🔗 Apply AWS migration phases: Assess, Mobilize, and Migrate and Modernize to plan and execute a structured migration strategy
- 🔗 Analyze different migration approaches, including Rehosting, eplatforming, and Refactoring, and determine their suitability for different scenarios
- 🔗 Implement AWS migration tools such as DMS, Snowball, Data Sync, SCT, CloudFormation, and Terraform, and validate data integrity through monitoring and troubleshooting

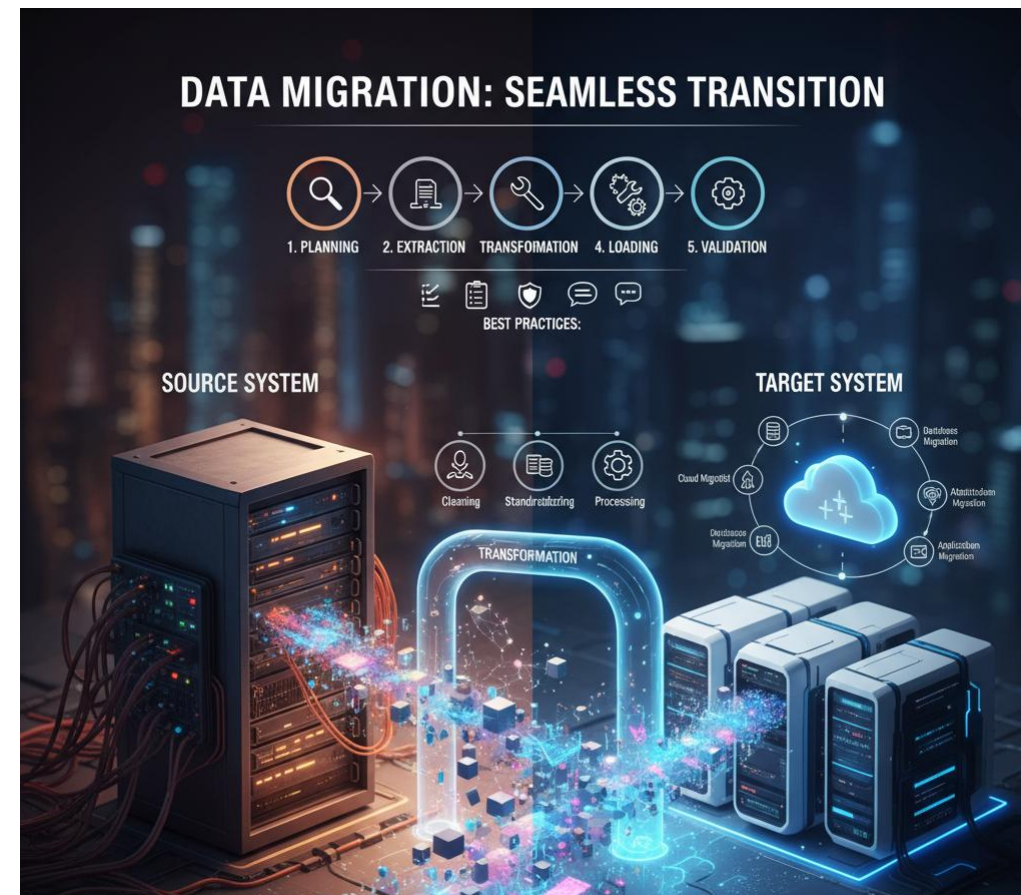




Understanding the Basics of Data Migration

Data Migration: Overview

It is the reliable transfer of information across platforms, formats, or environments to ensure accuracy, consistency, and usability without disrupting business operations.



Data migration ensures that data is transformed, cleaned, and validated for accuracy and usability in the target system, not just moved.

Data Migration: Advantages and Disadvantages

Advantages

- Improved performance
- Modern technology adoption
- Centralized data management
- Enhanced compliance reporting
- Sustained business continuity

Disadvantages

- Increased complexity
- Higher data risks
- Potential downtime
- Costly implementation process
- Time-intensive execution

Data Migration: Types

It is the process of transferring data between systems, storage types, or environments, playing a pivotal role in modern IT transformations. The following are the types:

Storage migration

Moving data between storage systems (for example, on-premises to cloud, legacy hardware to SSDs)

Data base migration

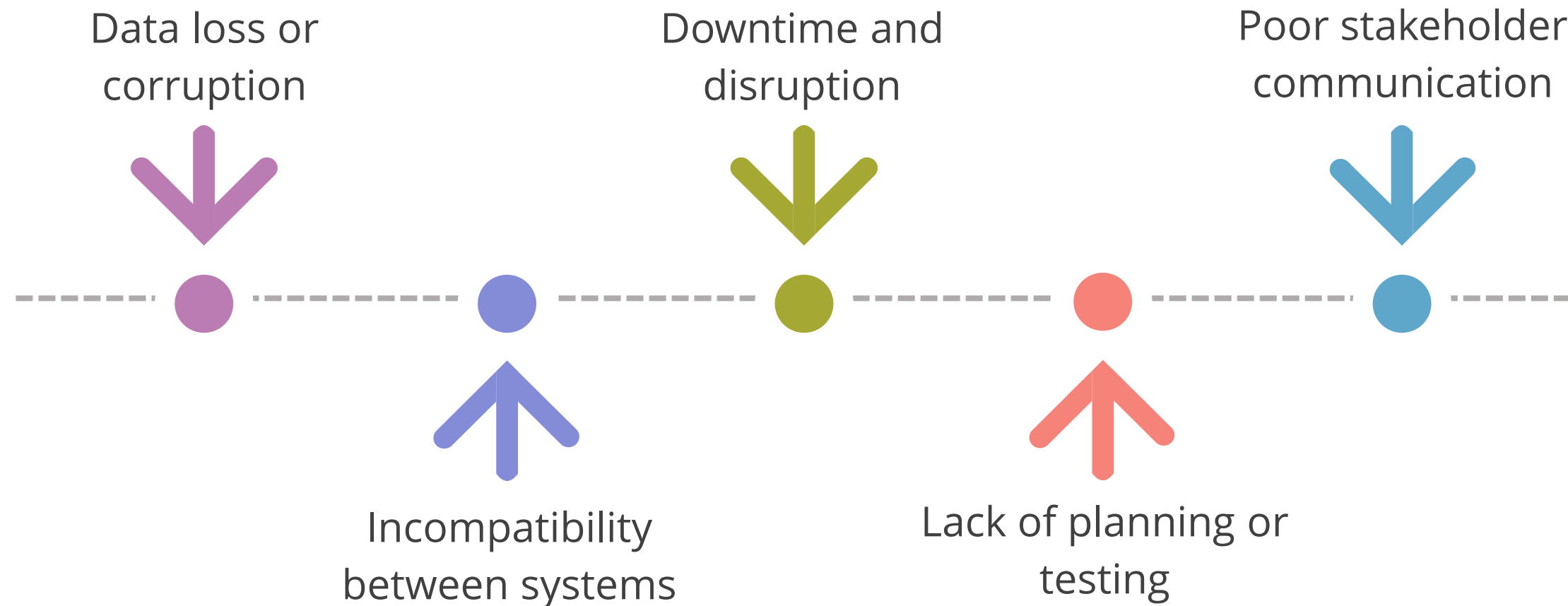
Transferring data between database systems (for example, Oracle to PostgreSQL) or upgrading within the same platform

Application or data replication

Duplicating data across systems in real time or on a schedule for high availability, disaster recovery, or analytics

Data Migration: Risks and Challenges

Despite its importance, data migration comes with several risks that can disrupt operations or result in significant losses if not properly managed. They are:



Data migration challenges can disrupt the entire process, so they must be identified early and addressed proactively.

Traditional vs. Cloud Migration

Traditional migration

- On-premises infrastructure
- Dependence on physical hardware (servers, networks, storage)
- High upfront investment in hardware and maintenance
- Limited flexibility and scalability
- Long provisioning times for new resources

Cloud migration

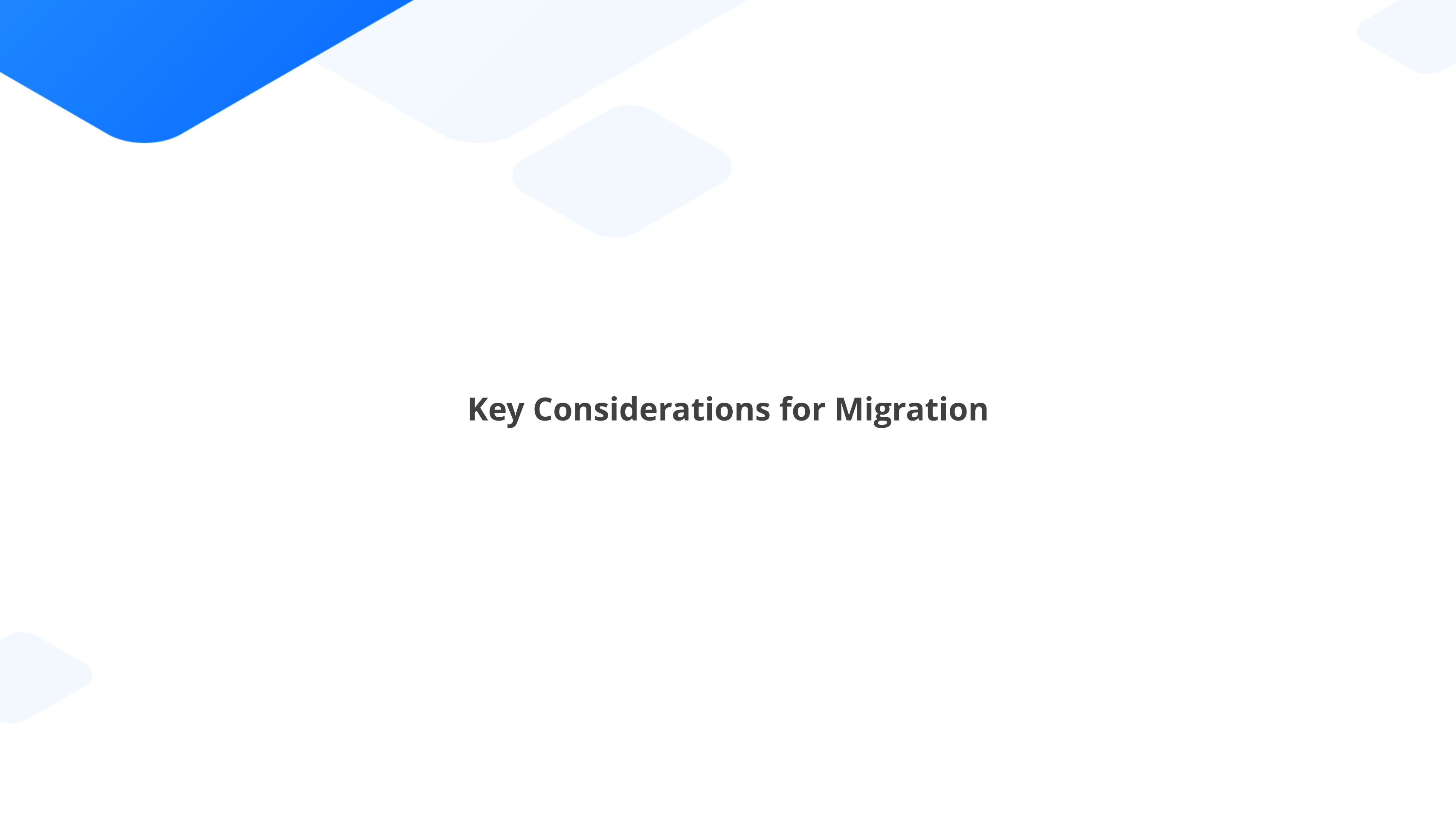
- Hosted and managed infrastructure by a cloud provider
- Pay-as-you-go pricing (eliminates large upfront capital expenses)
- Near-infinite scalability and elasticity
- Access to advanced services (AI, ML, analytics, DevOps tools)
- Built-in high availability and disaster recovery

Quick Check



A retail company is shifting from on-premise to the cloud. The IT manager wants flexibility, scalability, and cost savings. Which option best shows how cloud migration differs from traditional migration?

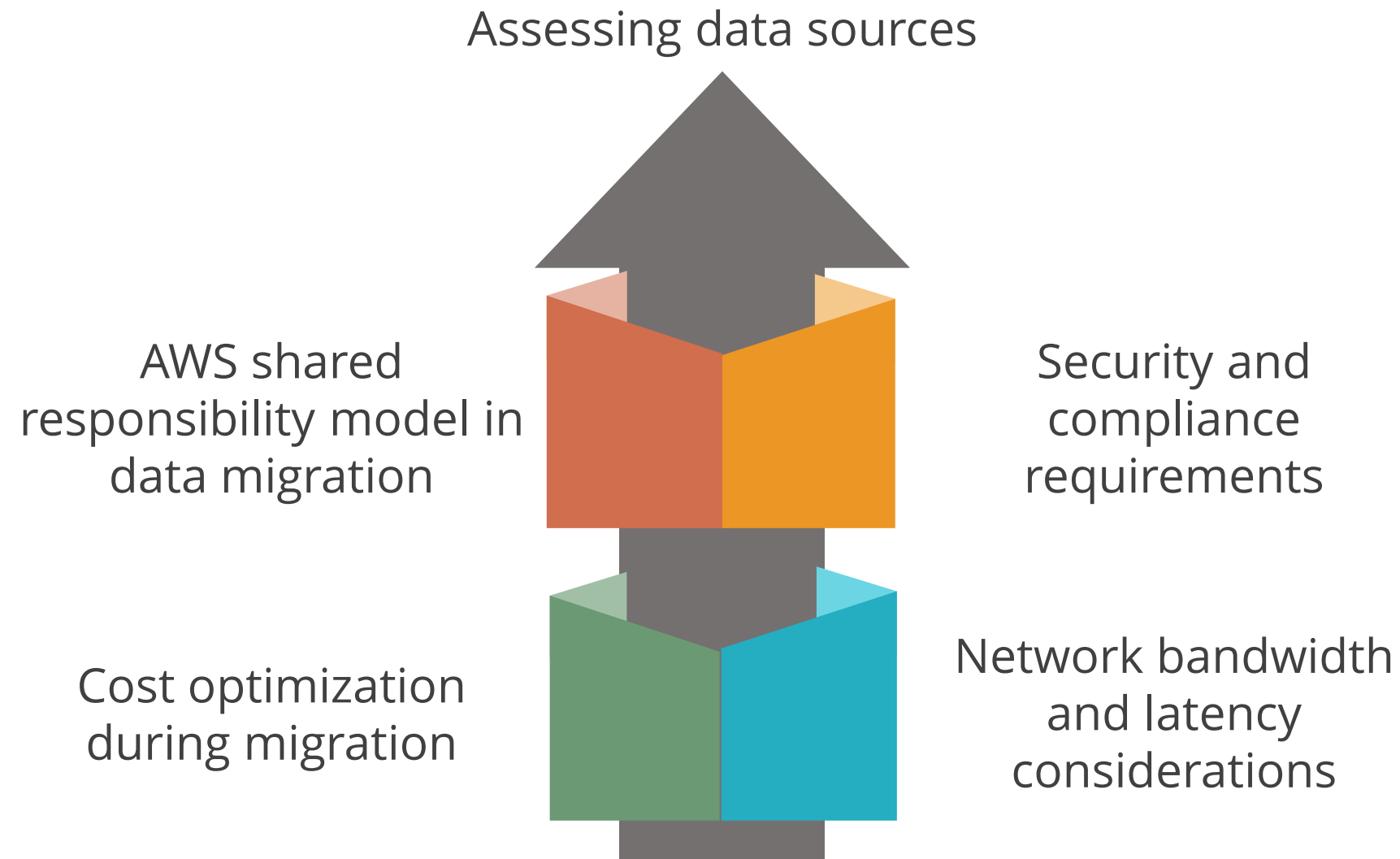
- A) Traditional migration uses object storage, while cloud migration does not.
- B) Cloud migration offers elasticity, scalability, and pay-as-you-go pricing models, unlike traditional approaches.
- C) Traditional migration always includes automation and monitoring tools.
- D) Cloud migration requires no planning or compliance checks.



Key Considerations for Migration

Core Factors in Cloud Data Migration

Migrating data to the cloud especially to platforms like AWS is a complex process that involves more than simply transferring files. The following are the key considerations:



Assessing Data Sources

A successful migration begins with a complete understanding of your data landscape. This involves categorizing data into structured and unstructured types:

Structured data

- Highly organized and stored in relational databases
- Contains customer details, transaction records, and inventory lists
- Easily searchable, filterable, and analyzable using tools like SQL

Unstructured data

- Lacks a fixed format and is not organized like relational data
- Contains emails, videos, images, PDFs, logs, and IoT sensor outputs
- Commonly stored in object storage solutions such as Amazon S3

Security and Compliance Measures

To ensure security and meet compliance requirements, organizations must implement the following key measures:

Encryption

Protect sensitive data in transit and at rest

Access control

Restrict access to authorized users only

Auditing and monitoring

Track activities, detect anomalies, and ensure accountability

Data residency

Comply with regional regulations on where data can be stored and processed

Network Bandwidth and Latency Considerations

To prevent slow performance, organizations should focus on three key considerations:

Bandwidth management

Plan for capacity and evaluate how bandwidth limits affect migration performance

Latency sensitivity

Assess the impact of delays on time-critical applications and mitigate accordingly

Optimized transfer tools

Use specialized tools and protocols to accelerate and secure large-scale data movement

Cost Optimization During Migration

Cloud migration can deliver significant long-term savings, but costs may spike if not carefully managed. The following are key tools and approaches for cost optimization during migration:

AWS pricing models

- Use flexible, pay-as-you-go pricing for short-term workloads
- Commit to 1-3 year plans for up to 75% savings
- Leverage spare AWS capacity for low-cost options

Storage cost optimization

- Select the right S3 tier
- Automate file management using lifecycle policies
- Track and analyze usage with cost explorer and AWS budgets

Other cost tools

- Apply cost-saving and security recommendations
- Use rightsizing suggestions for EC2 instances
- Forecast costs before migration to avoid surprises

AWS Shared Responsibility Model in Data Migration

In the cloud, security is a shared responsibility: AWS secures the infrastructure of the cloud, while you are responsible for protecting your data and applications in the cloud.

What AWS secures (Security of the cloud)

- Physical infrastructure, data centers, and networking
- Underlying hardware, storage, and compute resources
- Managed services and infrastructure-level security

What you must secure (Security in the cloud)

- Data confidentiality and encryption
- User permissions and access management
- Application-level security, including firewalls and patches

Quick Check



A healthcare company is migrating both structured records and unstructured imaging data to AWS. It must meet HIPAA compliance, handle limited bandwidth, and keep migration costs optimized. What should be the top considerations?

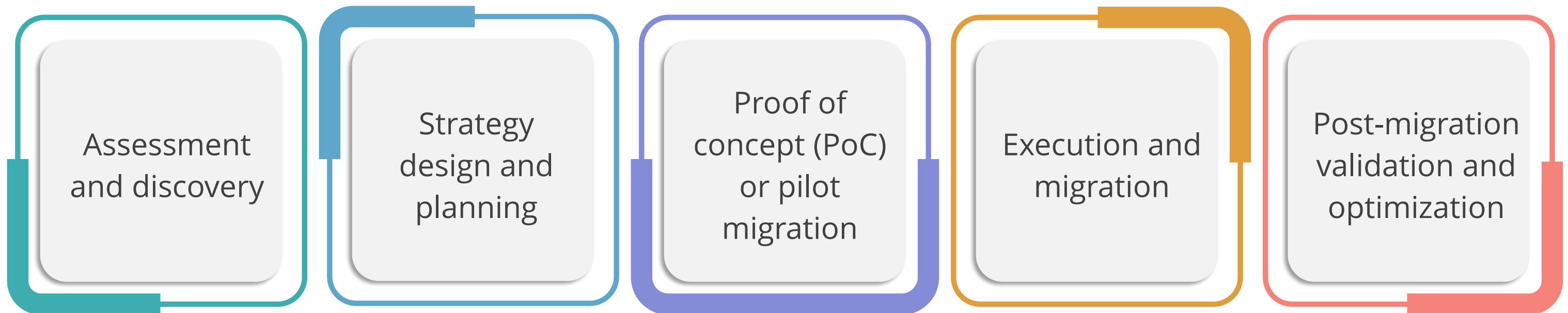
- A) Transfer only structured data first
- B) Ensure compliance, manage bandwidth, and use AWS pricing models
- C) Rely on AWS fully for compliance and security
- D) Migrate unstructured data later after cost review



Data Migration Strategies and Best Practices

Planning and Executing a Migration Strategy

Successful cloud migration requires a structured, step-by-step approach. The following stages provide a roadmap:



Assessment and Discovery

The following are the key steps in the assessment and discovery phase of migration:

Inventory systems and applications

Catalog databases, servers, applications, storage, and networking components

Classify data and workloads

Identify dependencies, usage patterns, criticality, and security sensitivity of each asset

Evaluate current performance

Detect system bottlenecks, capacity issues, and legacy infrastructure limitations

Define business goals

Clarify objectives such as cost reduction, scalability, or modernization to shape the migration plan

Strategy Design and Planning

Rehosting (lift and shift)

Move applications as-is to the new environment with minimal changes

Replatforming

Apply minor optimizations during migration, such as switching databases or upgrading operating systems

Refactoring or rearchitecting

Redesign applications to leverage new environment features like cloud-native microservices

Retiring or retaining

Identify systems to decommission or keep on-premises

Prioritization

Decide migration order, starting with low-risk, non-critical systems

Target environment selection

Choose AWS, Azure, hybrid, or private cloud based on performance, cost, compliance, and scalability needs

Proof of Concept (PoC) or Pilot Migration

It reduces risks by validating tools, testing performance, and uncovering issues before full-scale execution. Goals include:

Validate selected tools and migration techniques

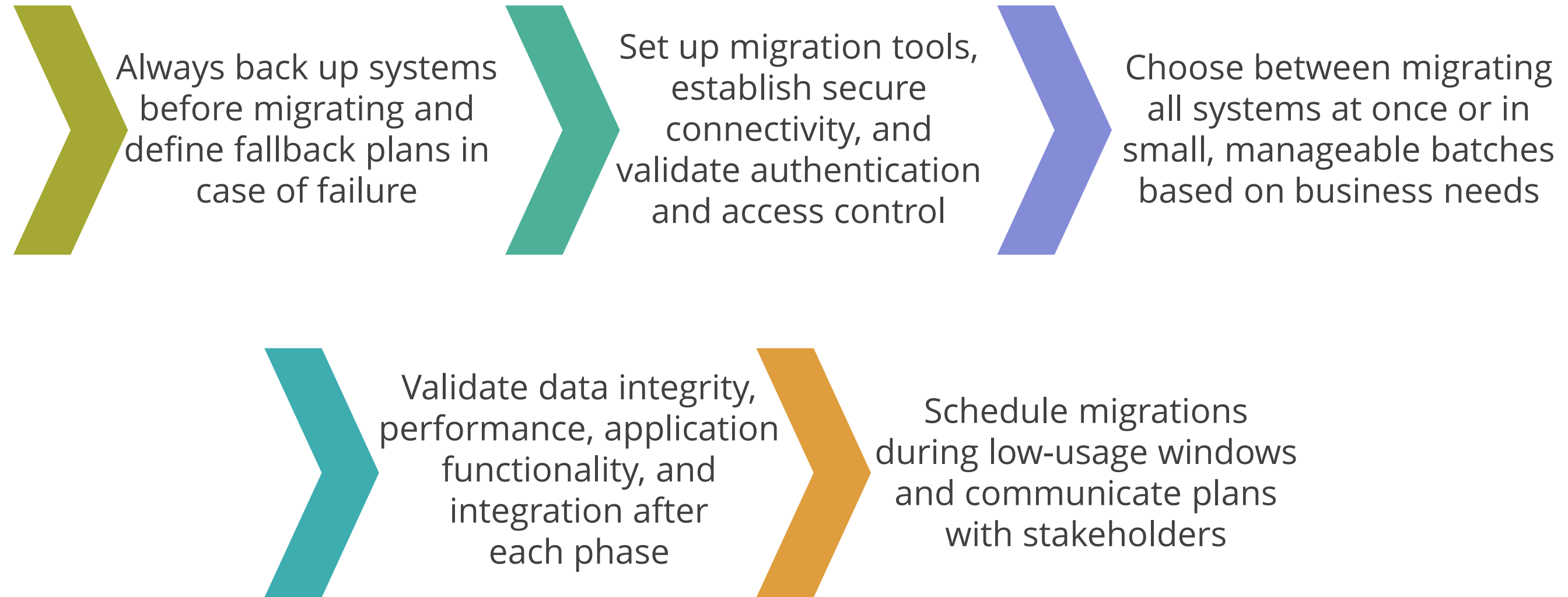
Identify application compatibility issues

Test network, bandwidth, and performance in real conditions

Adjust migration timelines and resource requirements

Execution and Migration

The following are the key steps for successful execution and migration:



Post-Migration Validation and Optimization

After migration, validating system performance and optimizing resources is critical to ensure long-term success and cost efficiency. The following are the key steps:

Retest application workflows, data integrity, and overall system performance

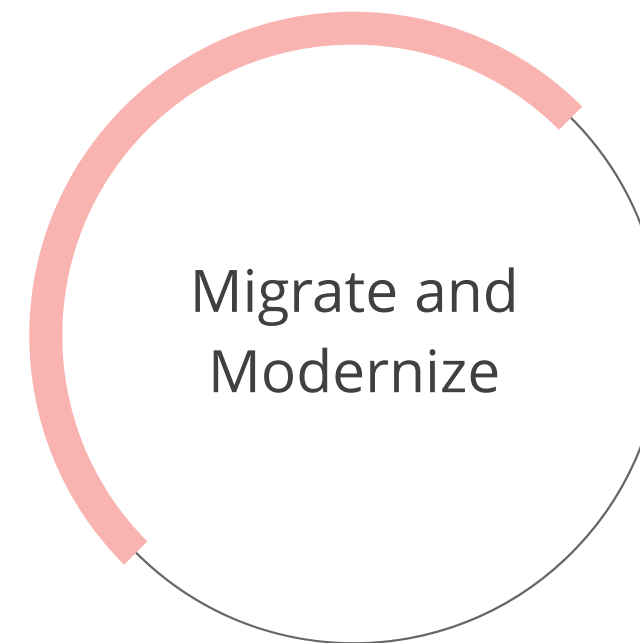
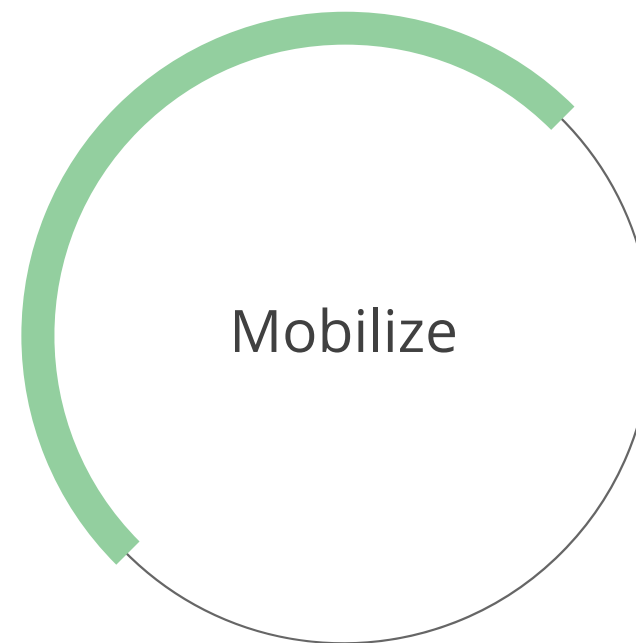
Set up dashboards and alerts with tools like CloudWatch, Azure Monitor, or third-party solutions to track availability and usage

Verify IAM roles, encryption settings, firewall rules, and compliance with policies

Optimize resource utilization, right-size instances, and apply pricing models like reserved or spot instances

AWS Migration Phases

AWS defines a structured migration framework to minimize risk and maximize efficiency. The following are the three core phases:



Phase 1: Assess – Understanding Your Readiness

This phase focuses on evaluating your organization's cloud readiness, identifying capability gaps, and defining what success looks like by:

- Conducting a thorough readiness assessment of business drivers, IT environment, and organizational capability
- Using the Cloud Adoption Readiness Tool (CART) to identify gaps in skills, operations, security, and governance
- Defining success criteria and align IT and business teams

Deliverables: Cloud readiness report, business case, IT-business alignment, and a high-level migration strategy

Phase 2: Mobilize – Building a Secure and Scalable Foundation

This phase establishes the operational and security foundation needed for migration, including landing zones, governance, and detailed migration plans by:

- Setting up a migration landing zone with governance, identity, and access controls
- Building a detailed migration plan with prioritized workloads and timelines
- Leveraging AWS tools like Migration Hub to manage and track migration progress.

Deliverables: Migration landing zone, migration plan, prioritized workloads, and pilot migrations

Phase 3: Migrate and Modernize – Moving and Evolving Workloads

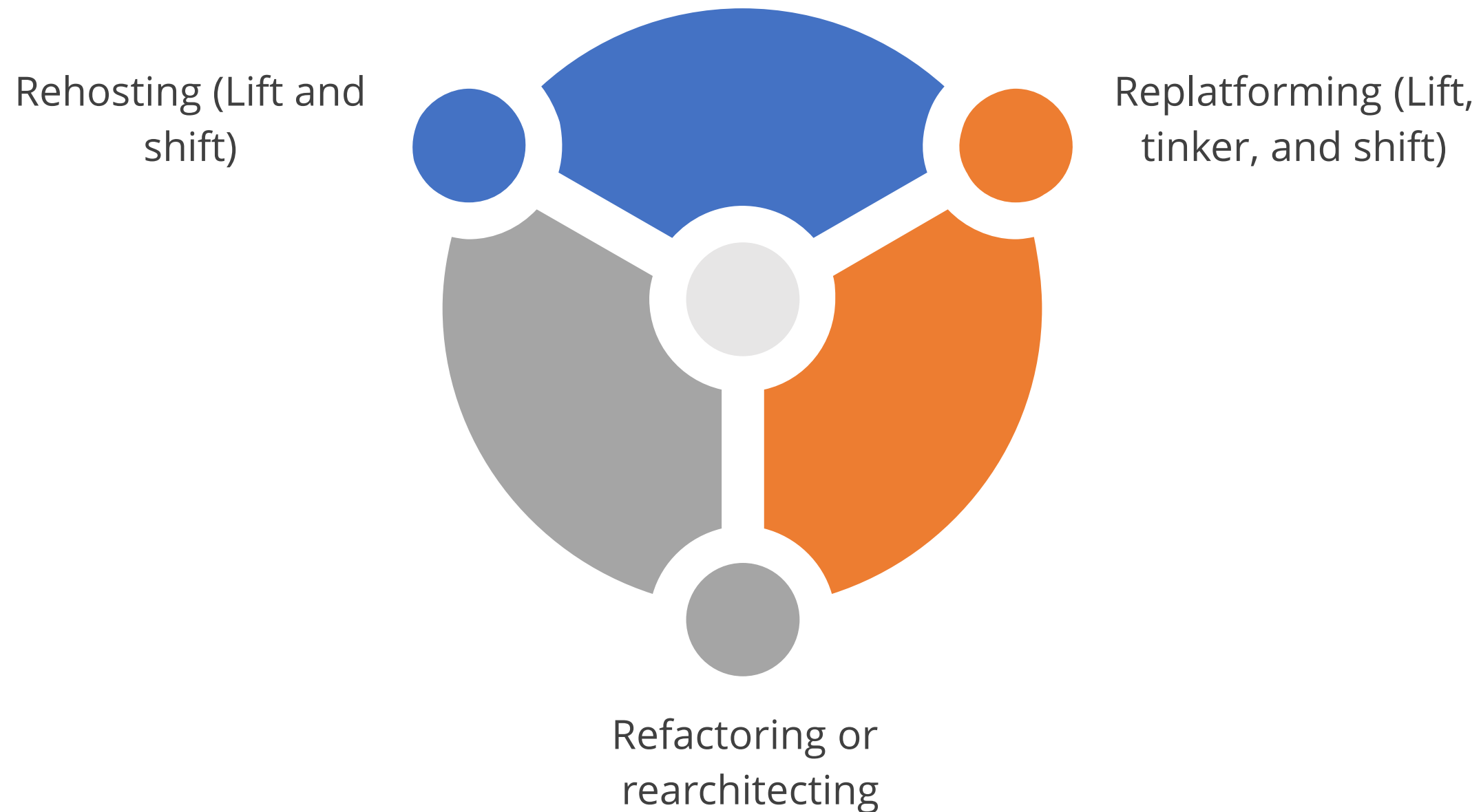
This phase involves moving workloads to the cloud, applying modernization strategies, and optimizing systems for long-term efficiency by:

- Migrating applications, databases, and workloads using AWS migration tools
- Applying modernization approaches like replatforming, refactoring, or containerization
- Optimizing performance, costs, and security post-migration

Deliverables: Cloud-hosted workloads, modernized applications, and optimized operations

Choosing a Migration Approach

When selecting a migration approach, organizations can choose from different strategies depending on business needs and cost. The following are the three main technical migration approaches:



Rehosting (Lift and Shift)

It means moving applications to the cloud with minimal or no changes, keeping their core setup intact.

How it works

- Lift the application and shift it to a cloud VM
- Keep OS, code, and configuration mostly unchanged
- Minimal adjustments during migration

Best for

- Fast migration of multiple applications
- Limited time and resources
- First step in a broader cloud strategy

Replatforming (Lift, Tinker, and Shift)

It makes small adjustments to applications, so they run more efficiently in the cloud without a full rebuild.

How it works

- Lift the application into the cloud
- Tinker with minor changes (for example, move to managed DB)
- Shift and deploy in the new environment

Best for

- Organizations seeking quick cloud benefits
- Applications needing moderate optimization
- Teams avoiding full redevelopment

Refactoring or Rearchitecting

It involves redesigning and rearchitecting applications to fully leverage cloud-native services and features.

How it works

- Rewrite application code for the cloud
- Use microservices, containers, or serverless
- Build scalable, resilient, cloud-native systems

Best for

- Core business applications
- Long-term modernization initiatives
- Apps requiring high scalability and performance

Use Case

Danske Bank's Hyper-Automated AWS Migration: 50% Faster, 25 PB Modernized



Problem

- Danske Bank managed 16,600 on-prem servers, 1,000+ applications, and 25 PB of data, making operations costly and complex.
- Traditional migration methods projected 5-8 years to complete, risking tech obsolescence and compliance exposure.
- Manual runbooks and fragmented team coordination led to errors, delays, and inconsistent results.

Use Case

Danske Bank's Hyper-Automated AWS Migration: 50% Faster, 25 PB Modernized



Solution

- The bank adopted an AWS Cloud migration strategy, following the phases of Assess, Mobilize, Migrate and Modernize.
- A hyper automation engine was developed using AWS Lambda, step functions, and DynamoDB, which automated 90% of migration tasks.
- The organization trained over 700 employees in cloud governance and compliance under the AWS shared responsibility model.

Use Case

Danske Bank's Hyper-Automated AWS Migration: 50% Faster, 25 PB Modernized



Outcome

- The migration timeline was accelerated by 50%, significantly reducing the overall duration of the program.
- Runbook creation time dropped from several weeks per application to just four seconds, streamlining execution and reducing human error.
- Infrastructure costs and operational overhead were reduced, allowing resources to focus on resilience, modernization, and innovation.

Quick Check



A financial services company has completed the Assess phase of its AWS migration journey. The team is now preparing the environment before moving any workloads. What is the primary goal of the Mobilize phase in this situation?

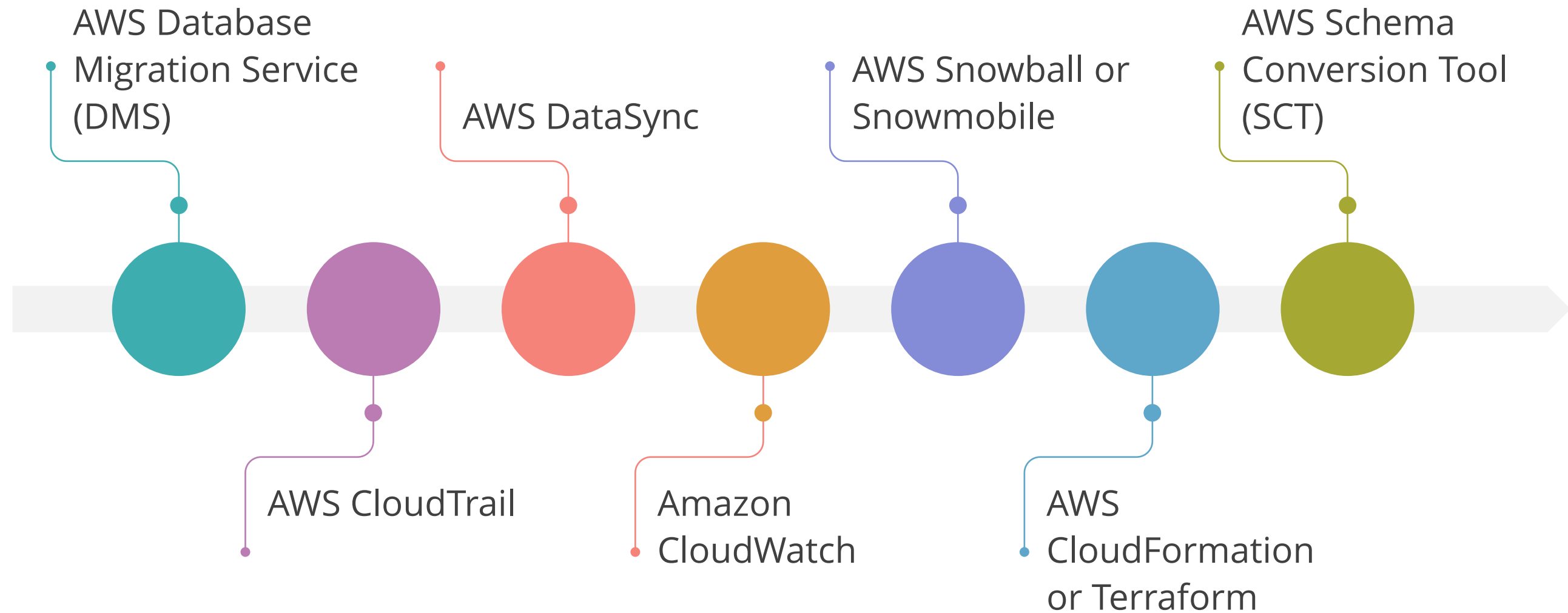
- A) Begin migrating databases to RDS
- B) Define business KPIs for cloud investment
- C) Set up a secure and governed environment using Landing Zone
- D) Use AWS Snowball for bulk data transfer



Tools and Techniques for Effective Data Migration

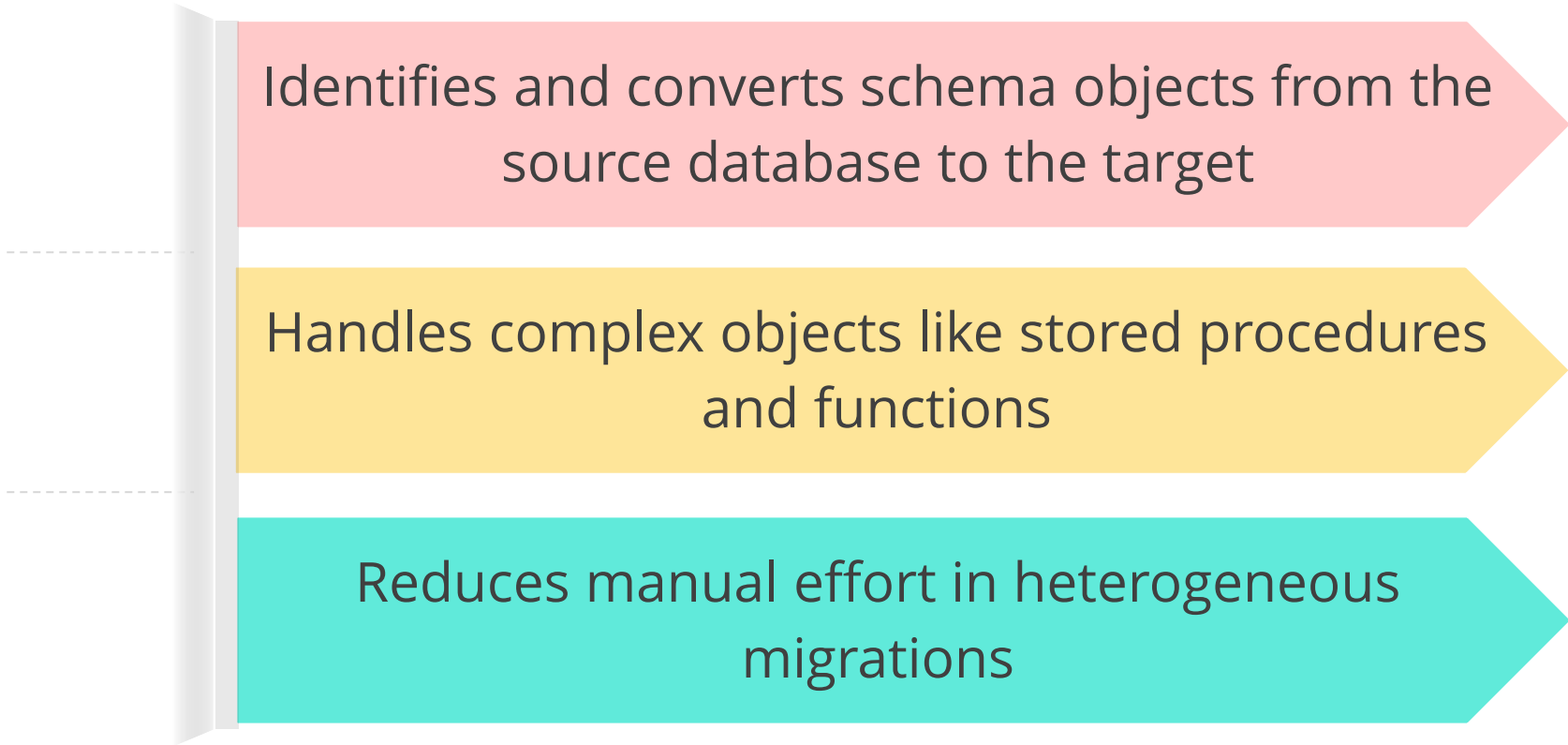
Tool and Techniques: Overview

The following are the key tools and techniques used in migration:



AWS Database Migration Service

It simplifies the process of moving databases to AWS with minimal downtime, supporting both homogeneous and heterogeneous migrations. The following are its key functions:



Identifies and converts schema objects from the source database to the target

Handles complex objects like stored procedures and functions

Reduces manual effort in heterogeneous migrations

Pros: Minimal downtime and wide support for different databases

Cons: Not a full ETL tool and may need manual work for complex transformations

AWS DataSync

It simplifies and accelerates data transfer between on-premises storage and AWS services, supporting hybrid and large-scale migrations. The following are its key functions:

Supports hybrid migrations across on-premises and cloud storage

Automates tasks like data validation and integrity checks

Provides secure transfer with encryption and built-in monitoring

Pros: Enables fast, automated, and reliable transfers with encryption and monitoring

Cons: Can be costlier than alternatives, and performance depends on network quality.

AWS Snowball or Snowmobile

It provides physical devices to transfer massive amounts of data when internet-based transfer is too slow or impractical. The following are its key functions:

Provides secure, rugged devices for petabyte-scale transfer

Offers containerized exabyte-scale migration

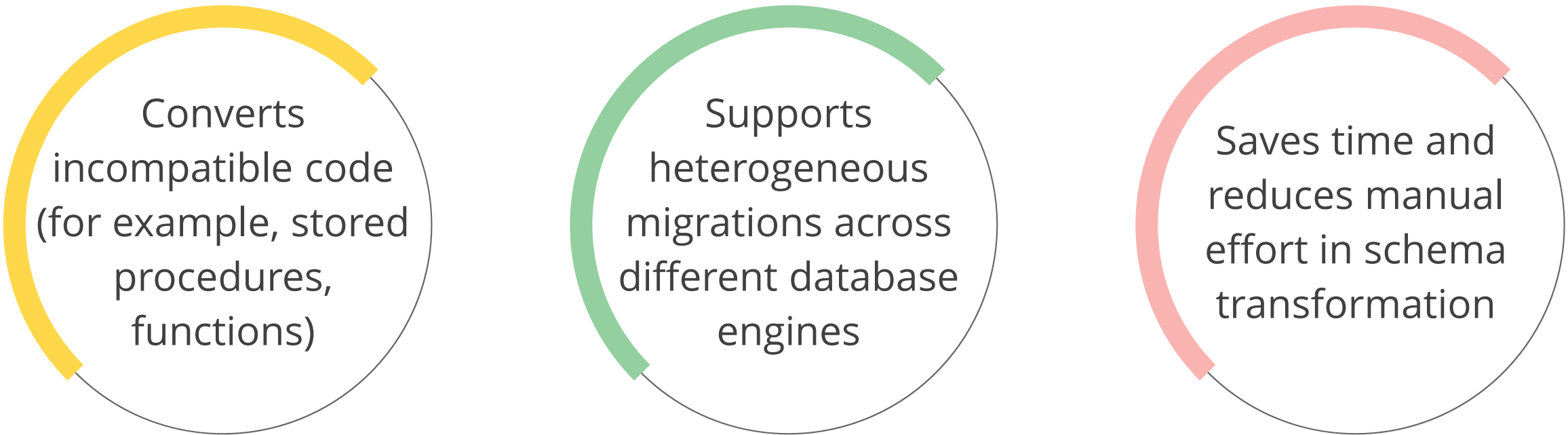
Bypasses slow or limited network bandwidth

Pros: Bypasses network limits and is highly secure

Cons: Logistics cause delays, making it unsuitable for real-time transfers

AWS Schema Conversion Tool (SCT)

It simplifies heterogeneous database migrations by automatically converting incompatible schemas, procedures, and functions for the target environment. The following are its key functions:



Converts incompatible code (for example, stored procedures, functions)

Supports heterogeneous migrations across different database engines

Saves time and reduces manual effort in schema transformation

Pros: Automates complex processes and saves significant time

Cons: May not convert 100% of code and often requires manual adjustments

AWS CloudTrail

It records and audits all API calls and activities within your AWS account. The following are its key functions:

Provides detailed logs for security, auditing, and compliance

Helps track user activity and changes across services

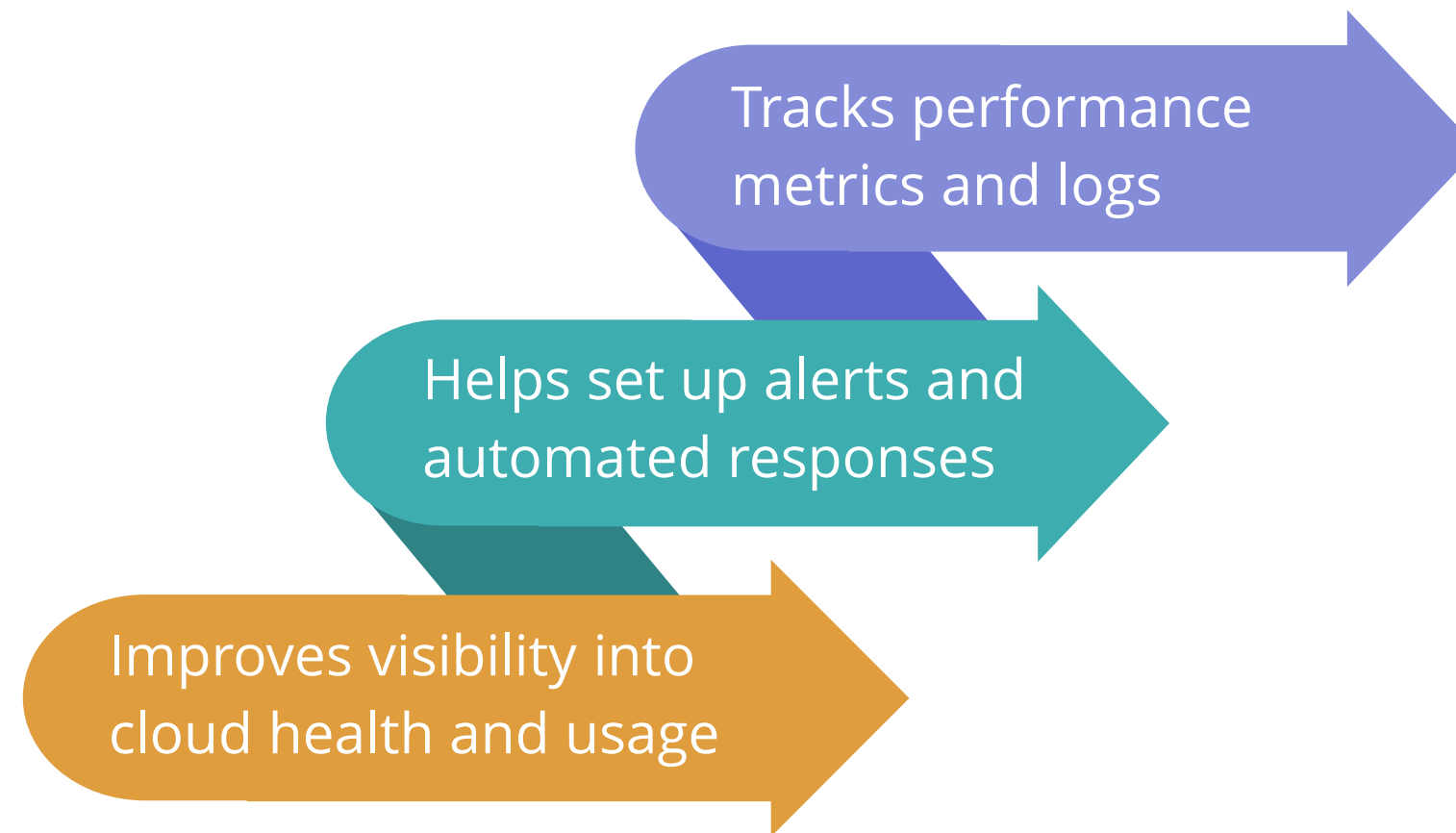
Supports forensic analysis in case of incidents

Pros: Enhances accountability and strengthens compliance

Cons: Generates large log volumes that require structured management

Amazon CloudWatch

It provides real-time monitoring and alerting for AWS resources and applications. The following are its key functions:



Pros: Enables proactive monitoring and faster issue resolution

Cons: Generates large data volumes, which can increase cost and complexity

AWS CloudFormation or Terraform

It automates and standardizes cloud deployments using Infrastructure as Code (IaC). The following are its key functions:

Defines infrastructure in code for repeatable, consistent deployments

Enables automation, version control, and scalability

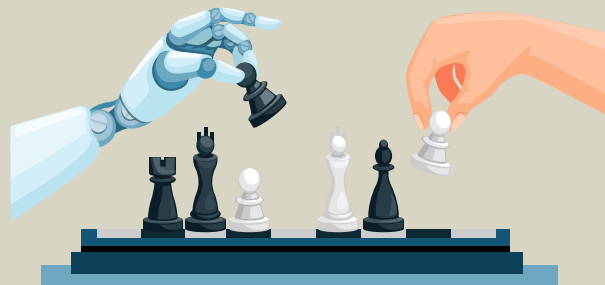
Reduces manual errors by applying pre-defined templates

Pros: Ensures consistency, automation, and fewer errors in deployment

Cons: Requires a steep learning curve and can become complex at large scale

Use Case

Aardwark Heterogeneous Database Migration



Problem

- Aardwark needed to migrate its mission-critical Oracle databases to Amazon Aurora PostgreSQL to reduce licensing costs and improve scalability.
- The migration involved heterogeneous environments, making schema conversion complex and error-prone.
- Manual migration posed risks of downtime, data inconsistency, and long project timelines.

Use Case

Aardwark Heterogeneous Database Migration



Solution

- The team used the AWS Schema Conversion Tool (SCT) to analyze Oracle schemas, convert them to PostgreSQL, and identify elements requiring manual intervention.
- They applied the AWS Database Migration Service (DMS) to replicate data and enable change data capture (CDC) for near-zero downtime.
- Infrastructure was automated with Terraform, streamlining deployment and ensuring consistent environments.

Use Case

Aardwark Heterogeneous Database Migration



Outcome

- The migration was completed successfully with minimal downtime, ensuring business continuity.
- The company reduced licensing and operational costs by moving away from proprietary Oracle workloads.
- Aardwark achieved scalable, cloud-native operations on Aurora PostgreSQL, enabling faster innovation and resilience.

Quick Check



A healthcare company needs to migrate petabytes of medical records from its on-premises data center to AWS. The transfer must be secure and efficient, but the existing network bandwidth cannot handle such large volumes in a reasonable timeframe. Which AWS migration tool is best suited for this scenario?

- A) AWS Database Migration Service (AWS DMS)
- B) AWS Snowball or Snowmobile
- C) AWS DataSync
- D) AWS Schema Conversion Tool (SCT)

Key Takeaways

- Data migration involves moving information across storage, databases, and applications, with clear distinctions between traditional and cloud approaches.
- Successful cloud migration requires assessing data sources, ensuring compliance, managing network and costs, and applying AWS's shared responsibility model.
- AWS migration framework follows three structured phases: Assess, Mobilize, and Migrate and Modernize.
- Migration approaches include Rehosting, Replatforming, and Refactoring, chosen based on business needs.
- AWS migration tools like DMS, Snowball, DataSync, SCT, CloudFormation, CloudWatch, and CloudTrail support secure, efficient, and well-governed migrations.



Migrating Database Using AWS DMS

Duration: 25 Mins.

Project agenda: To perform a structured migration of data from a MySQL Community Edition RDS instance to an Amazon Aurora MySQL cluster using AWS Database Migration Service (DMS). This process involves provisioning and configuring the source RDS instance and the target Aurora cluster, establishing secure connectivity between them, and setting up AWS DMS components, including the replication instance, source and target endpoints, and migration tasks.

Description: You are tasked with performing a database migration from a MySQL Community Edition RDS instance to an Amazon Aurora MySQL instance using AWS Database Migration Service (DMS). This project involves setting up the source and target endpoints, configuring replication tasks, and ensuring secure connectivity between the databases. The goal is to minimize downtime during migration, validate schema and data consistency, and simulate real-world replication scenarios. By using AWS DMS, this setup provides a hands-on demonstration of cloud-native database migration practices, helping you understand how to move workloads to Aurora while maintaining application availability.



Migrating Database Using AWS DMS

Duration: 25 Min.

Perform the following:

1. Create MySQL and Aurora DB instances in the RDS service
2. Create a replication instance in AWS DMS
3. Implement database migration using AWS DMS

Expected deliverables: A fully configured source MySQL RDS instance and target Aurora MySQL cluster, a detailed AWS DMS setup guide covering prerequisites, IAM roles, endpoints, replication instances, and tasks, a successfully executed replication task performing full load and ongoing data replication, a validation report confirming schema and data consistency between source and target, documented observations on migration performance and downtime considerations, and a final migration report summarizing the entire process with identified issues, applied fixes, and recommendations for production-grade database migrations.



Q&A

